# INTELIGÊNCIA ARTIFICIAL

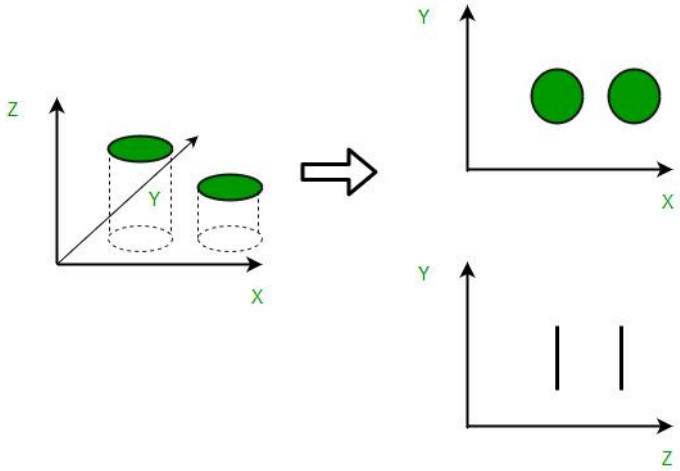
## Aprendizado Descritivo
## Clustering

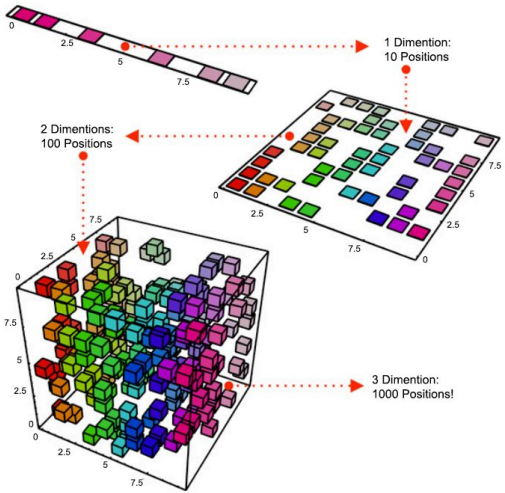
**Marcos de Souza**
mso2@cesar.school

C.E.S.A.R school

# Curse of Dimensionality

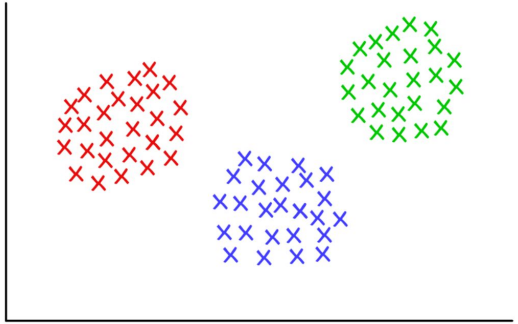
- Além da dificuldade na **visualização dos grupos**, a alta dimensionalidade aumenta a **dispersão dos dados** e com isso prejudica a performance dos algoritmos
- Outro problema é o custo de armazenamento de uma quantidade excessiva de informação (exemplo, imagens de altíssima resolução)

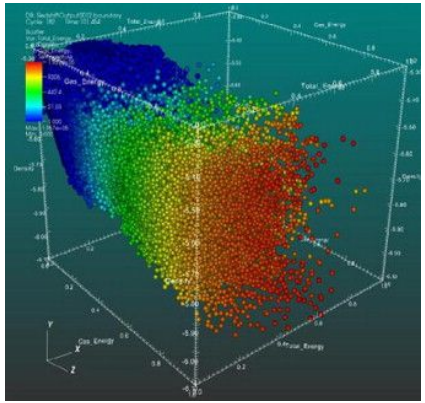


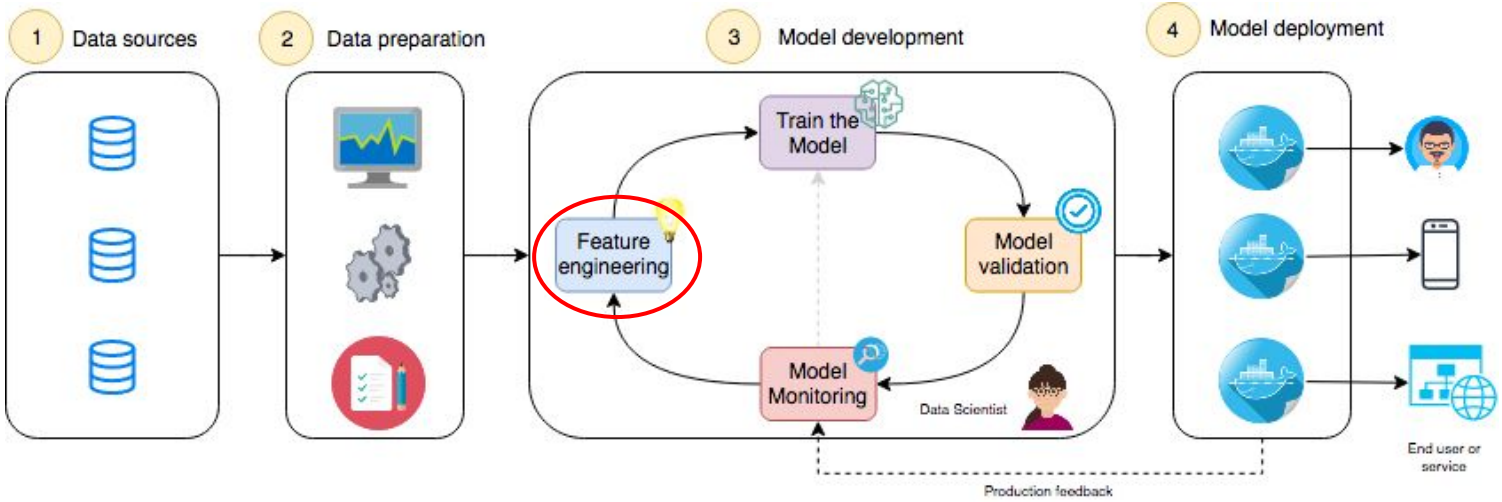
# Curse of Dimensionality

- Após aplicarmos uma medida de distância para todos os exemplos buscamos encontrar a **formação natural de grupos** (imagem da esquerda).
- No entanto, quando lidamos com um dataset de alta dimensionalidade pode ocorrer de não haver uma **separação clara entre os grupos** (imagem da direita).
- Para isso precisamos aplicar técnicas de redução de dimensionalidade antes de qualquer análise de grupos.





# Data Science Solution Workflow




```python
# fit the model
model.fit(X_train, y_train)

# save the model
import pickle
pickle.dump(model, open("model.pkl", "wb"))

# load the model
model = pickle.load(open("model.pkl", "rb"))

# use model to predict
y_pred = model.predict(X_input)
```

**mas antes ...**

# Transformações

- Quando os dados estão em escalas diferentes é comum **normalizarmos** para converter em uma mesma escala (exemplo: valores entre 0 e 1)
- Essa **normalização** diminuirá o **impacto** no cálculo de distância entre os objetos
- Os dados em escalas distintas geralmente são bem dispersos quando aplicamos uma medida simples de distância

| | Pclass | Sex | Age | SibSp | Parch | Ticket | Fare | Embarked |
|---|---|---|---|---|---|---|---|---|
| **0** | 3 | 1 | 22.0 | 1 | 0 | 523 | 7.2500 | 2 |
| **1** | 1 | 0 | 38.0 | 1 | 0 | 596 | 71.2833 | 0 |
| **2** | 3 | 0 | 26.0 | 0 | 0 | 669 | 7.9250 | 2 |
| **3** | 1 | 0 | 35.0 | 1 | 0 | 49 | 53.1000 | 2 |
| **4** | 3 | 1 | 35.0 | 0 | 0 | 472 | 8.0500 | 2 |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... |
| **886** | 2 | 1 | 27.0 | 0 | 0 | 101 | 13.0000 | 2 |
| **887** | 1 | 0 | 19.0 | 0 | 0 | 14 | 30.0000 | 2 |
| **888** | 3 | 0 | 24.0 | 1 | 2 | 675 | 23.4500 | 2 |
| **889** | 1 | 1 | 26.0 | 0 | 0 | 8 | 30.0000 | 0 |
| **890** | 3 | 1 | 32.0 | 0 | 0 | 466 | 7.7500 | 1 |

891 rows × 8 columns

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| **0** | 1.0 | 1.0 | 0.271174 | 0.125 | 0.000000 | 0.769118 | 0.014151 | 1.0 |
| **1** | 0.0 | 0.0 | 0.472229 | 0.125 | 0.000000 | 0.876471 | 0.139136 | 0.0 |
| **2** | 1.0 | 0.0 | 0.321438 | 0.000 | 0.000000 | 0.983824 | 0.015469 | 1.0 |
| **3** | 0.0 | 0.0 | 0.434531 | 0.125 | 0.000000 | 0.072059 | 0.103644 | 1.0 |
| **4** | 1.0 | 1.0 | 0.434531 | 0.000 | 0.000000 | 0.694118 | 0.015713 | 1.0 |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... |
| **886** | 0.5 | 1.0 | 0.334004 | 0.000 | 0.000000 | 0.148529 | 0.025374 | 1.0 |
| **887** | 0.0 | 0.0 | 0.233476 | 0.000 | 0.000000 | 0.020588 | 0.058556 | 1.0 |
| **888** | 1.0 | 0.0 | 0.296306 | 0.125 | 0.333333 | 0.992647 | 0.045771 | 1.0 |
| **889** | 0.0 | 1.0 | 0.321438 | 0.000 | 0.000000 | 0.011765 | 0.058556 | 0.0 |
| **890** | 1.0 | 1.0 | 0.396833 | 0.000 | 0.000000 | 0.685294 | 0.015127 | 0.5 |

891 rows × 8 columns

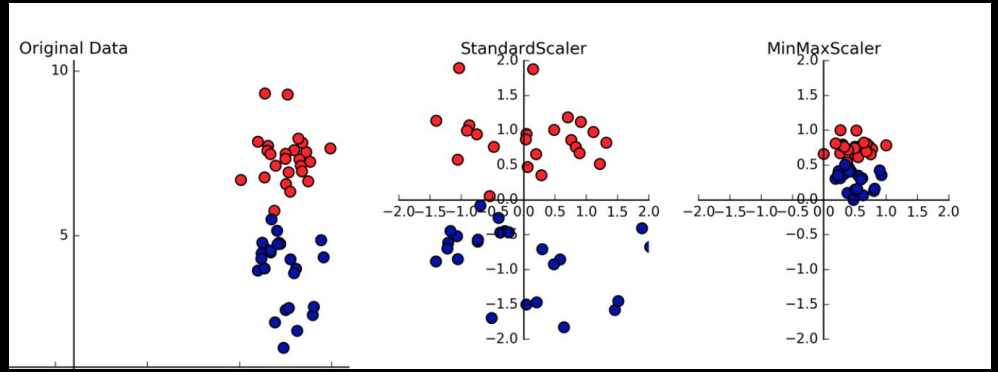
# Scaler em Python

Code use example

```
# importa o pacote cluster
from sklearn.preprocessing import MinMaxScaler, StandardScaler

scaler = MinMaxScaler() # StandardScaler()

# Aplica a normalização
X_new = scaler.fit_transform(X)
```

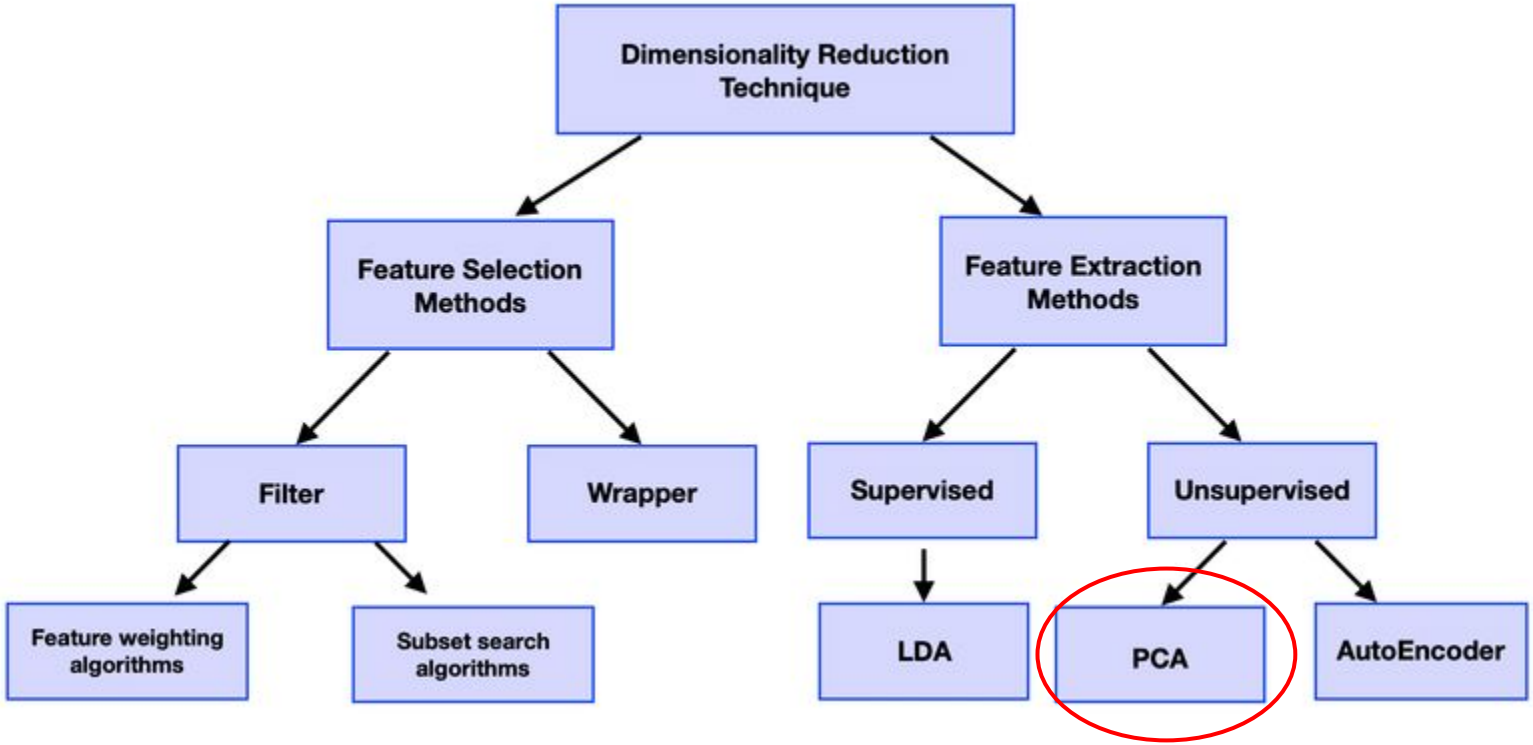


# Métodos de Redução de Dimensionalidade

- Como podemos **transformar** os dados de forma **supervisionada** e **não supervisionada** ?

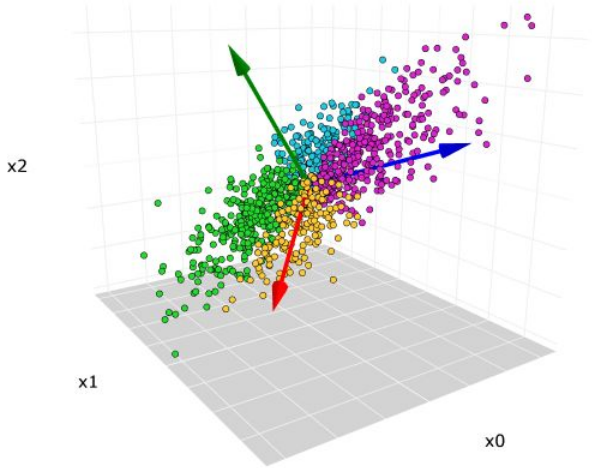
# Redução da Dimensionalidade

Uma visão **resumida** das técnicas de redução de dimensionalidade



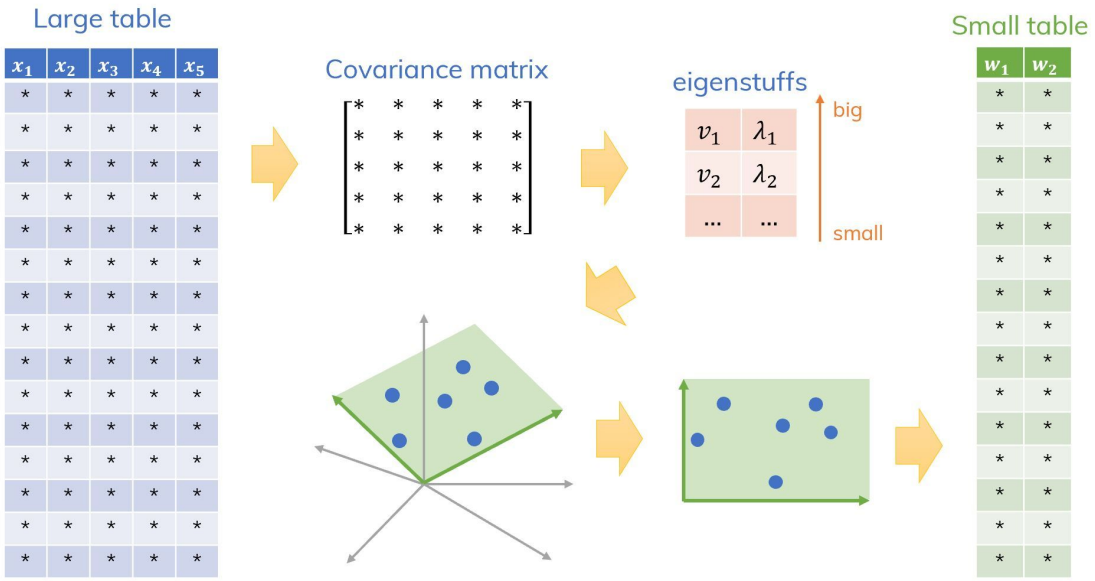
# PCA

- Trata-se de uma técnica não-supervisionada de transformação que visa **linearizar** a direção dos dados
- Possui uma versão não-linear onde é preciso informar uma função de kernel (inclusive linear) que irá convergir com os dados



# PCA step-by-step

- passo 1: calcula a matriz de **covariância**
- passo 2: extrai os **autovetores** a partir dessa matriz
- passo 3: **seleciona** os autovetores mais explicativos

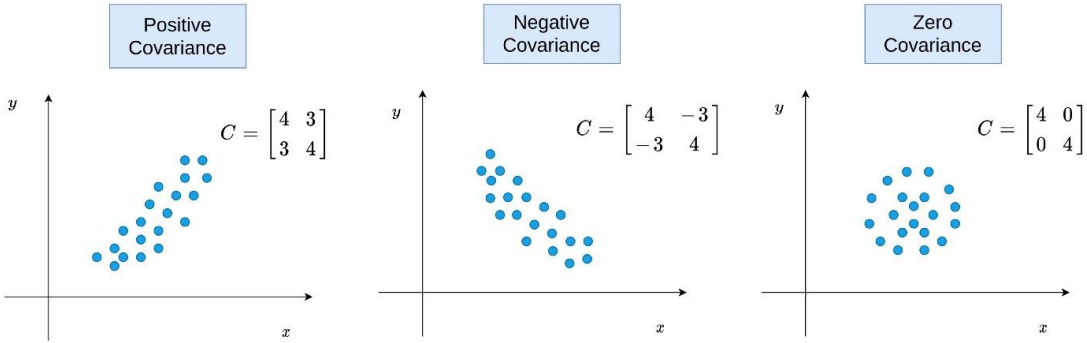


# Matriz de Covariância

Ao calcularmos a matriz de covariância encontramos a direção dos dados

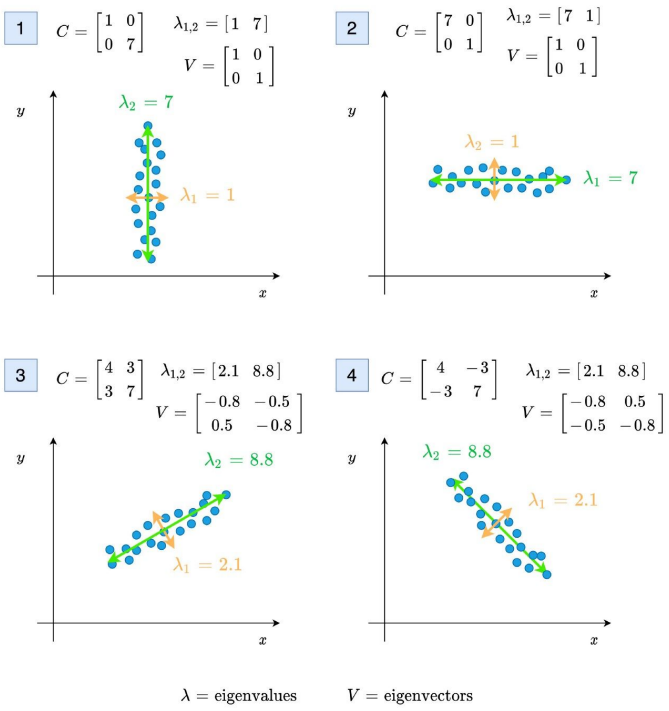
$$\mathrm{Cov}\,(X, Y) = \frac{\sum(X_i - \overline{X})(Y_j - \overline{Y})}{n}$$

$$\begin{bmatrix} \mathrm{Var}(x_1) & \cdots\cdots & \mathrm{Cov}(x_n, x_1) \\ \vdots & \ddots & \vdots \\ \mathrm{Cov}(x_n, x_1) & \cdots\cdots & \mathrm{Var}(x_n) \end{bmatrix}$$



# Autovetores da Matriz de Covariância

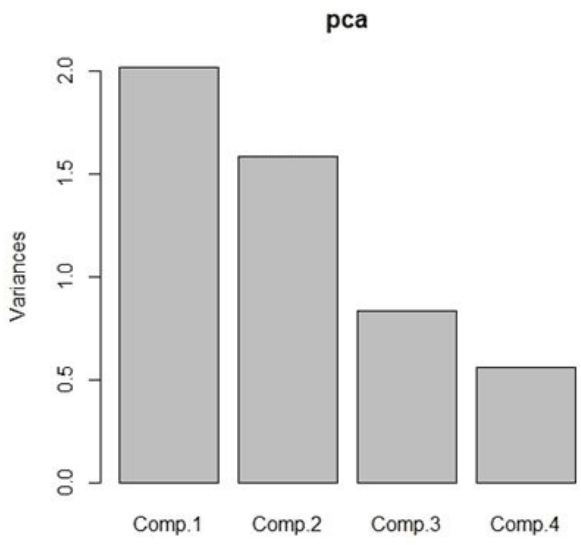
- Ao calcularmos a matriz de covariância encontramos a direção dos dados
- Quando calculamos os autovetores achatamos (**linearizamos**) a dispersão dos dados

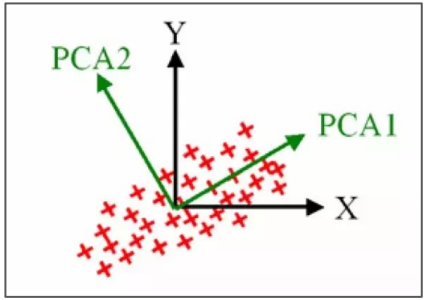


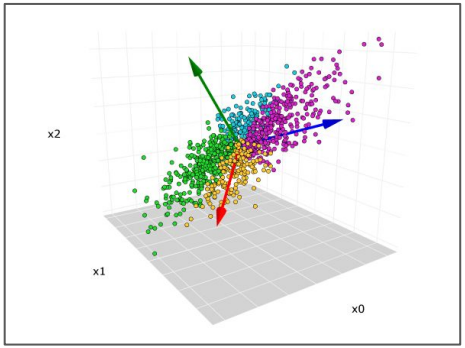

Ref:
https://towardsdatascience.com/5-things-you-should-know-about-covariance-26b12a0516f1

# Selecionando os Componentes Principais

- Plotamos os pontos em todas as dimensões e selecionamos os autovetores (componentes) que tenham maior cobertura sobre a variabilidade dos dados (variância)

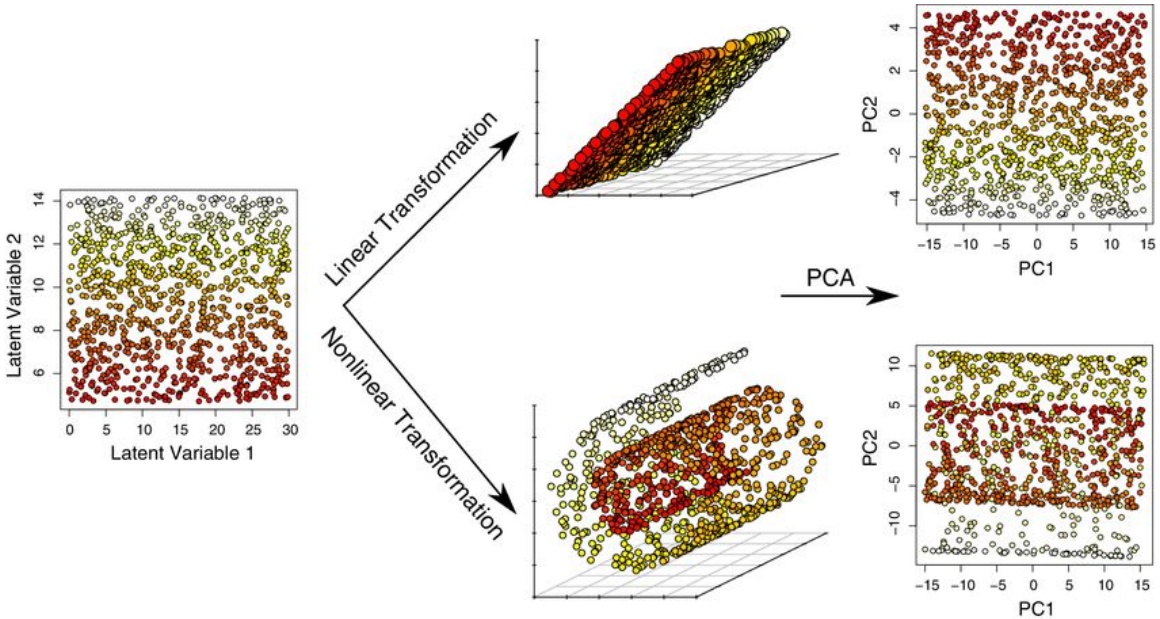


# Limitações

- Em estruturas complexas não lineares o PCA obterá baixo desempenho na detecção dos clusters/classes.
- Neste ponto é importante selecionarmos um kernel apropriado



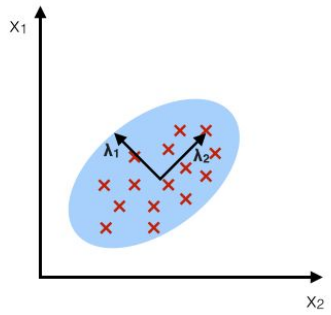
1

# Análise de Discriminante Linear (LDA)

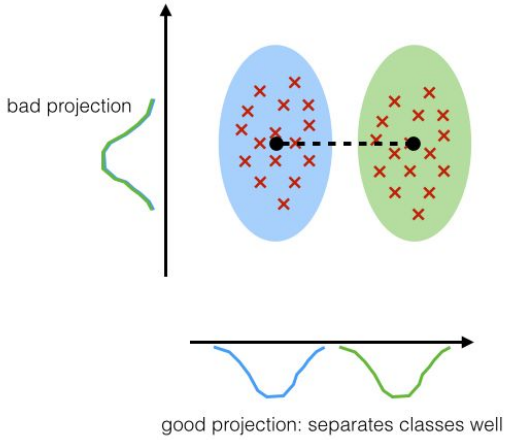
# Selecionando os Componentes Principais

Trata-se de um método supervisionado de geração de componentes lineares que consigam maximizar a separabilidade dos dados

**PCA:**
component axes that maximize the variance

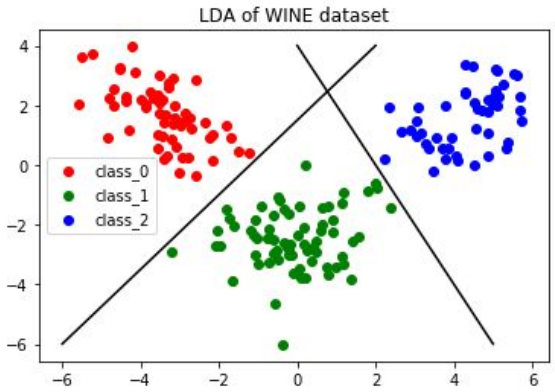


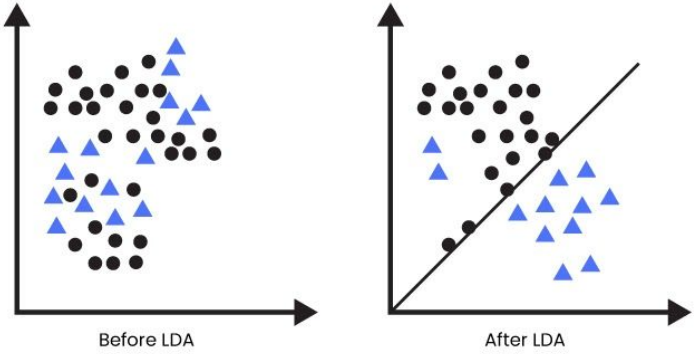

**LDA:**
maximizing the component axes for class-separation

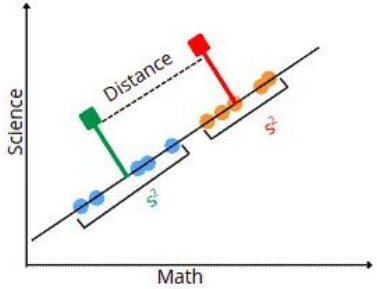


# Análise de Discriminante Linear

Diferente do PCA, o número de componentes no LDA será o número de labes -1



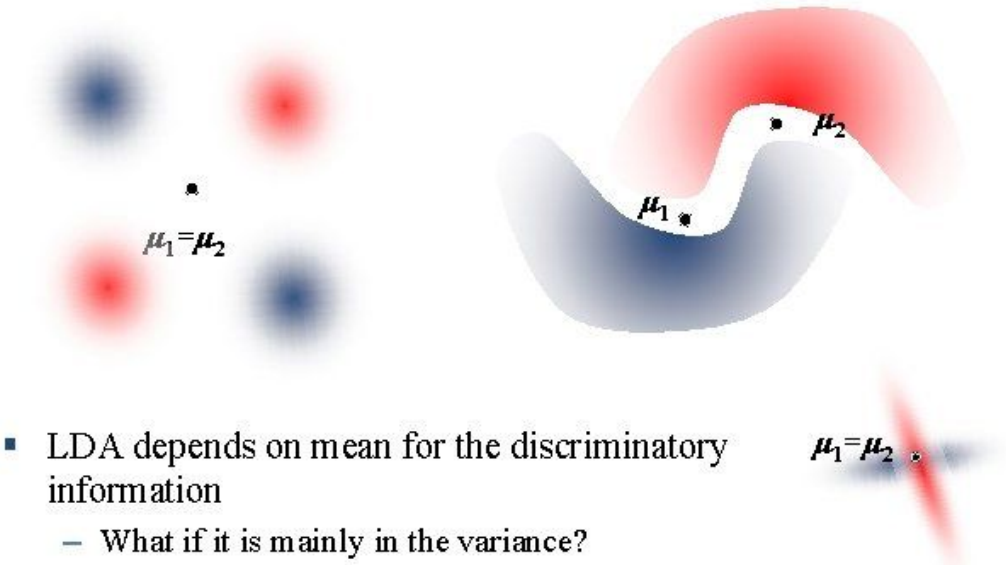




$$\frac{(\text{mean1} - \text{mean2})^2}{s^2 + s^2}$$

# Análise de Discriminante Linear

Limitações: o LDA presume que os dados pertencem a uma distribuição gaussiana



# PCA em Python

Code use example

```
# importa o pacote
from sklearn.decomposition import PCA

reductor = PCA(n_components=2)

# Aplica a redução
X_reduced = reductor.fit_transform(X)
```

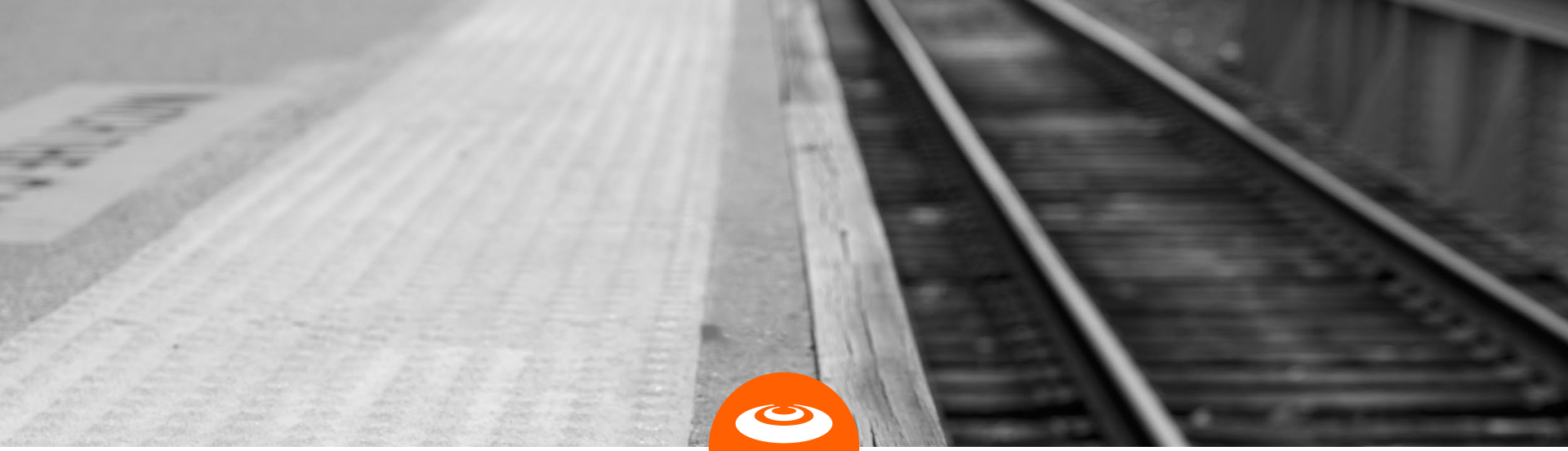
# LDA em Python

Code use example

```
# importa o pacote
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

reductor = LinearDiscriminantAnalysis()

# Aplica a redução
X_reduced = reductor.fit_transform(X, y_true)
```



# Hands On!

Experimentos em Machine Learning com Python

C . e . S . A . R

Pessoas impulsionando inovação.

Inovação impulsionando negócios.

**NOSSO CONTATO**

mso@cesar.org.br
mso2@cesar.school